

Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN

ANONYMOUS AUTHOR(S)

As AI-RAN platforms run radio processing and AI inference on the same GPUs, neural receivers create GPU work that schedulers cannot see. Until its neural receiver (NeuralRx) finishes, an uplink transport block (TB) may still need a second decoding by the conventional receiver. This *pending recovery* is in no GPU queue, but it becomes mandatory if the receiver fails. This paper presents Backstop, a GPU scheduler that tracks pending recoveries explicitly. Backstop gives GPU time to AI only when an independent verifier confirms that every TB could still be recovered even if every pending receiver failed. It then keeps each decision valid on the GPU: it checks a latest start time before each launch, applies all receiver results at once, and frees GPU time only after the GPU reports completion. On two four-GPU A100 nodes under a validated test timeline, Backstop has 0 safety violations in 120 boundary tests, while admitting AI into idle GPU time fails 40/40. Across 13,716 feasible settings of a capacity model, Backstop admits more AI than static reservation in 26.2% of them, and its p99 decision time is 1.494 ms with 64 pending recoveries.

CCS Concepts: • **Computer systems organization** → **Distributed architectures; Real-time systems; Networks** → *Network resources allocation*.

Additional Key Words and Phrases: AI-RAN, GPU scheduling, neural receiver, conditional recovery, MPS, admission control

ACM Reference Format:

Anonymous Author(s). 2027. Backstop: Recovery-Safe GPU Sharing Between Neural Receivers and AI Inference in AI-RAN. In *Proceedings of ACM SIGMETRICS 2027 (SIGMETRICS '27)*. ACM, New York, NY, USA, 29 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Radio access networks (RANs) increasingly run baseband processing on GPUs, and AI-RAN platforms run radio physical-layer (PHY) processing and AI inference on the same GPUs [3, 14, 19, 22]. To use the GPU time that radio processing leaves idle, these platforms add AI services, such as large language model (LLM) inference, next to deadline-critical PHY tasks [12, 21]. This sharing works only if every radio task still finishes before its deadline [7], and ordinary GPU sharing can disrupt 5G processing [19]. Neural receivers make this harder. A neural receiver replaces channel estimation, equalization, and demapping with a learned model, and whether it decodes a block is known only after it runs [6, 10]. However, the GPU schedulers of these platforms protect only the radio work that is already in a GPU queue or in a resource reservation.

The difficulty comes from how neural reception falls back. A neural receiver (NeuralRx) is optional work: if it succeeds in time, it delivers the uplink transport block (TB), and if it fails, is late, or loses its result, the conventional receiver must decode the same TB. Until the NeuralRx result is known, this recovery is in no GPU queue, yet it must still be able to finish before the radio deadline. When several cells run NeuralRx together, their recoveries compete for one conventional decoder, and their failures can happen together. In addition, NVIDIA Multi-Process Service (MPS) caps the GPU threads of each process, but it does not bound host scheduling, queueing, or process restarts [15]. Thus, a scheduler that fills idle GPU time with AI can push a future recovery past its deadline, and a scheduler that reserves every possible recovery wastes the GPU time that a successful NeuralRx frees.

Existing systems share accelerators between radio processing and other workloads in different ways. As Table 1 summarizes, they differ in what their schedulers protect or allocate. vRAN and AI-RAN resource managers [3, 7, 12, 19–22] allocate compute shares, queued requests, reserved

Table 1. What the closest public designs schedule.

System	What its scheduler protects or allocates	What its scheduler does not see
YinYangRAN, CloudRIC, Concordia	GPU share, queued job with a deadline, or reserved capacity	The pending recovery of a TB before it is queued
DARIS, REEF, XSched, nvtaskset	GPU stage, kernel, context, or partition	Radio recovery, buffer ownership, and one commit per TB in one checked schedule
ARCHES, OCUDU, SMEC	PHY model choice, timing class, or edge SLO	The worst-case recovery of all cells together with an AI slot
Backstop	The pending recovery of each TB	Tracked, checked before admission, and enforced until the GPU finishes

capacity, or worker placement between RAN functions and other workloads. GPU scheduling systems [4, 5, 9, 23] partition, prioritize, or preempt GPU stages, kernels, and contexts. PHY and edge platforms [16, 18, 25] choose PHY models, expose timing classes inside the distributed unit (DU), or coordinate edge service-level objectives (SLOs). However, all of them schedule resources or queued jobs. Their public designs neither represent the recovery that a neural receiver may need nor keep an admission decision valid until the GPU finishes, and a decision that is correct when made can become wrong before it runs.

In this paper, we present Backstop, a recovery-safe GPU scheduler that shares AI-RAN GPUs between neural receivers and AI inference. Backstop records each recovery that may still be needed as a *pending recovery* and accepts a decision only when an independent verifier confirms that every TB could still be recovered if all pending receivers failed. Specifically, Backstop introduces 1) a scheduling model that decides together which TBs try NeuralRx, when recoveries run, and when AI runs, over one set of pending recoveries shared by several cuPHY instances, 2) enforcement that keeps each accepted decision valid on the GPU by checking a latest start time before each launch, applying all receiver results at once, and freeing GPU time only after the GPU reports completion, and 3) a prediction, based only on validated time bounds, that classifies each state as SAFE+AI, SAFE-ONLY, INFEASIBLE, or UNVALIDATED before it runs. Our evaluation on two four-GPU A100 nodes under a validated test timeline shows 0 safety violations in 120 boundary tests for Backstop and 40/40 for idle-time admission. When AI requests arrive in bursts taken from BurstGPT with a 200 ms deadline, Backstop serves 6.2-18.5% more AI tokens on time than recovery-first, its variant without AI-first order, and static reservation serves none, while idle-time admission misses the recovery deadline in up to 10.0% of periods. On one GPU, Backstop makes the same decisions, and early AI slots that share the GPU with NeuralRx serve 2.6-70.3% more on-time AI tokens without losing a timely NeuralRx result. In 1,200 periods on physical GPUs, the shared GPU finishes every period before its scheduled end, and under seven kinds of injected faults every TB commits exactly once before its result deadline. Across 13,716 feasible settings of a capacity model, Backstop admits more AI than static reservation in 3,600 (26.2%). The scheduler matches an exact checker on 16,023 states, and its p99 decision time is 1.494 ms with 64 pending recoveries. On the original trace, Backstop and recovery-first deliver the same 349,387 timely tokens.

2 Background and Motivation

2.1 AI-RAN on shared GPUs

AI-RAN platforms run RAN functions and AI services on the same GPUs [12, 14, 19–22]. In our setting, these GPUs run three kinds of work: optional neural reception, mandatory conventional reception, and external AI.

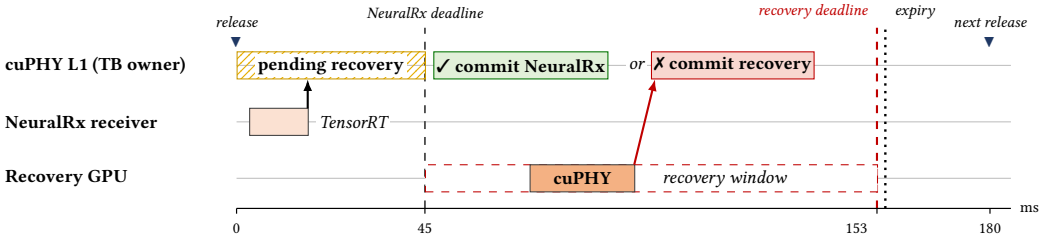


Fig. 1. One uplink TB in one 180 ms period of our test timeline, which is slower than 5G slot timing. Until the 45 ms NeuralRx deadline, the TB has a pending recovery that no GPU queue contains. If the NeuralRx result fails, is late, or is missing, one cuPHY recovery must finish before the 153 ms recovery deadline.

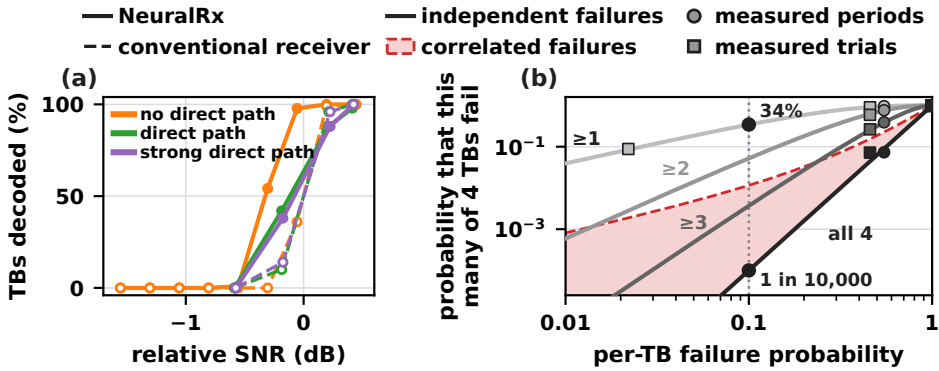


Fig. 2. The number of recoveries that a period needs is hard to predict. (a) Share of TBs that NeuralRx (solid) and the conventional receiver (dashed) decode in three channels: a synthetic fading channel without a direct path (block Rayleigh) and two 3GPP channels with a direct path (CDL-D) and a strong direct path (CDL-E); 0 dB is where the conventional receiver decodes half of the TBs. (b) Probability that one or more, two or more, three or more, or all four TBs of a period fail when each TB fails with probability p . Lines assume independent failures; the band shows the all-four event with a correlation of 0.3. Circles are measured periods, and squares are groups of four measured receiver trials.

Neural Reception. An uplink TB is decoded either by a conventional receiver, such as the cuPHY physical uplink shared channel (PUSCH) pipeline [14], or by a neural receiver [6, 10]. A cuPHY instance runs the PUSCH pipelines of several cells, and in our test timeline each cell sends one TB per period (Figure 1). A TB commits the NeuralRx result if it passes its cyclic redundancy check (CRC) by the NeuralRx deadline; otherwise, one conventional recovery must finish before the recovery deadline. Figure 2a shows that the NeuralRx result is hard to predict. In a synthetic fading channel without a direct path (block Rayleigh, 500 TBs per point) and in two 3GPP channels with a direct path (CDL-D and CDL-E [1, 11], 50 TBs per point), the NeuralRx success rate rises from near 0% to near 100% within 1 dB of the signal quality at which the conventional receiver decodes half of the TBs. In CDL-D and CDL-E, the conventional receiver also decodes some TBs that NeuralRx misses. Link adaptation targets a 10% block error rate [2], which keeps TBs near this transition.

GPU Sharing. MPS runs several CUDA processes on one GPU at the same time and caps the share of GPU threads that each process may use [15], but it does not bound host scheduling,

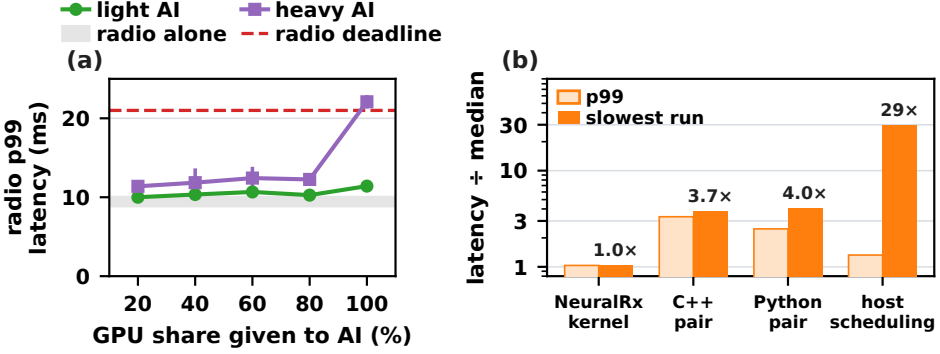


Fig. 3. Execution time varies on shared A100 GPUs. (a) Radio p99 latency of a two-cell, 273-PRB PUSCH pipeline against the GPU share of a co-running Qwen prefill: one 16-token request (light AI) or eight 512-token requests (heavy AI). Lines connect run means, and bars span the runs. (b) How much slower the p99 and the slowest run are than the median run: the NeuralRx GPU kernel (300 runs), a cross-GPU receiver pair on a partly native C++/CUDA path (300 runs) and on a Python path (1,000 runs), and host scheduling (1,200 periods).

queueing, process restarts, or memory allocation. Multi-Instance GPU (MIG) splits a GPU into isolated parts, but a fixed split cannot reuse the GPU time that a successful NeuralRx frees. Figure 3a runs Qwen2.5-1.5B prefill [17] next to a two-cell, 273-PRB PUSCH pipeline and varies the GPU share given to Qwen. With one 16-token request, the radio p99 latency stays within 11.9 ms at every share, against 8.9-10.0 ms without AI. With eight 512-token requests, it stays within 13.9 ms up to an 80% share but reaches 21.2-22.9 ms at 100%, where 93 of 1,000 slots miss the 21 ms deadline. Thus, a GPU share is safe only for the kind of AI work under which it was tested.

Execution Tails. Figure 3b compares the slowest runs with the median run. The NeuralRx TensorRT graph stays within 3.5% of its median, but end-to-end paths do not. At the 4.5 ms result deadline of the Aerial testMAC [14], cross-GPU receiver pairs take up to 4.0 and 3.7 times their median on a Python path and a partly native C++/CUDA path, and host scheduling takes up to 29 times its median. Thus, a schedule computed from GPU kernel times can become wrong before it runs.

Together, these observations tie radio correctness to AI admission decisions that are made before the receiver result is known. Existing systems split GPU resources between RAN and AI workloads [12, 19–22] or preempt GPU kernels [4, 9, 23], but their public designs do not schedule the recovery that neural reception may need.

2.2 Challenges in scheduling pending recovery

Existing schedulers admit AI based on the work in GPU queues or on a fixed reservation [7, 19, 20]. With neural reception, this approach faces three challenges: the number of needed recoveries is uncertain, the cells of several cuPHY instances share one GPU for recovery, and a decision must stay valid until it runs.

Pending Recovery. Before the NeuralRx result is known, the conventional recovery of a TB may or may not be needed: it is needed only if the NeuralRx fails. We call it a *pending recovery*. Figure 2b turns a per-TB failure probability p into the number of recoveries that a period with four TBs needs; measured periods follow the curves for independent failures. Admitting AI into idle GPU time reserves no recovery and is wrong whenever one recovery is needed, which happens in 34% of periods at $p = 0.1$. Static reservation holds four recoveries, which a period uses only when all four

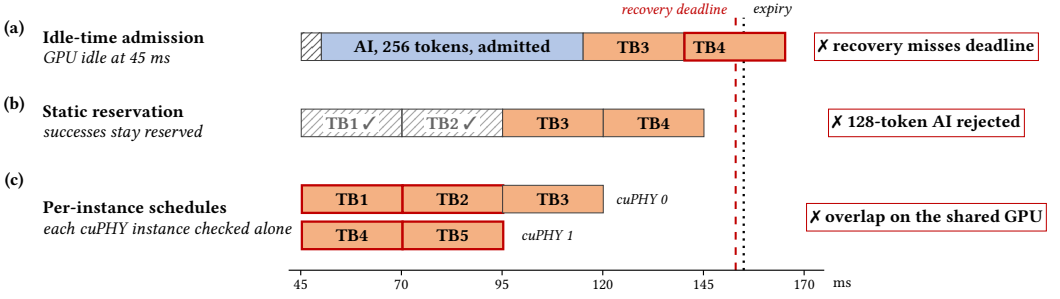


Fig. 4. Three ways existing policies fail after two timely successes (TB1, TB2). (a) Admitting AI into idle GPU time pushes a needed recovery past the recovery deadline. (b) Static reservation rejects a 128-token AI request that would fit once the successful TBs release their reservations. (c) Schedules checked per cuPHY instance claim the same GPU time.

TBs fail, once in 10^4 periods with independent failures. A failure correlation of 0.1 or 0.3 within a period makes this all-four event 16 or 116 times more likely. Thus, planning for the typical number of recoveries is unsafe, and holding every reservation until all results are known wastes the GPU time that successes free.

Scheduling Failures. Figure 4 shows three failures after two timely successes in our test timeline. Idle-time admission sees an empty queue at the 45 ms NeuralRx deadline, admits a 256-token AI request, and pushes a needed recovery past the recovery deadline. Static reservation keeps the reservations of the two successful TBs and rejects a 128-token AI request that would fit without them. Schedules checked per cuPHY instance are each correct alone, but they claim the same time on the shared recovery GPU. Thus, a scheduler must track pending recovery explicitly, replan whenever a result arrives, and check one schedule for all pending recoveries that share a GPU.

Execution Gaps. A schedule that is correct when made must stay correct when it runs. Table 2 lists seven failure cases that define this requirement. In our development runs, a 23.5 ms host stall launched a 128-token AI request outside its scheduled time, and applying simultaneous results one by one created an intermediate state that never existed. Thus, the scheduler must count launch overhead in every schedule, apply simultaneous results at once, and keep GPU time owned until the GPU reports completion. Section 3 describes how Backstop does this.

Table 2. Failure cases that schedule enforcement must cover.

Failure case	Wrong assumption	What Backstop does
Checked per instance	Schedules checked per cuPHY instance combine safely on one GPU	One schedule for all pending recoveries on the GPU
Slow dispatch	A checked schedule stays valid until launch	Recheck within a time budget and enforce a latest start time
Results applied one by one	Simultaneous results can be applied in any order	Apply all results of a deadline at once
AI slot without launch time	AI execution time includes launch overhead	Add a 5 ms launch budget to every AI slot
Reply lost after the effect	Retrying after an unclear outcome is harmless	New version on timeout; no AI without completion evidence
Two pending launch requests	An at-most-once RPC shows who owns the GPU time	At most one pending launch request
Warm bound reused	Worker restarts do not change time tails	Validate each bound under its measurement conditions

3 Backstop Design

Backstop treats pending recovery as explicit state and keeps each decision valid until the GPU finishes the work it started. This section presents the architecture, pending-recovery scheduling, enforcement on the GPU, and the scheduling algorithm, and then states what Backstop guarantees and how it is implemented.

3.1 Overall architecture

Backstop schedules pending recovery and external AI on the GPUs of one AI-RAN node. Figure 5 shows its architecture. A control layer on the host makes the decisions, and long-running GPU workers under MPS execute them. The figure follows one period through seven steps (❶-❷) in an example in which TB1 and TB2 succeed, TB3 fails its CRC, TB4 is late, and TB5 is rejected when it arrives.

Control Layer. One host coordinator keeps four objects. *Decision pages* record, for each TB, the NeuralRx result, the resulting action, and the version of the decision. *Pending recoveries* list every TB whose conventional recovery may still be needed, with its recovery window; in the example, TB3 and TB4 remain with the 45-153 ms window, while TB1 and TB2 are cancelled. The *shared-GPU schedule* holds the current schedule X_t of the shared GPU and a candidate X' that adds one AI slot, with its 5 ms launch budget, before the recovery deadline. The *independent verifier* commits X' or keeps X_t .

Execution Layer. Long-running MPS workers execute all GPU work. On the four-GPU node (Figure 5b), GPU0 and GPU1 run the cuPHY instances that own TB1-TB3 and TB4-TB5 with a 50% MPS share. GPU3 runs the TensorRT NeuralRx as a CUDA graph with an 80% share, receives IQ samples and channel estimates over NVLink, and returns log-likelihood ratios (LLRs). GPU2 is the shared GPU: it runs the cuPHY recovery worker with an 80% share and the Qwen worker with a 20% share, and it writes the completion markers. On one GPU (Figure 5a), the same processes share the device and exchange IQ, LLR, and TB buffers through CUDA IPC.

Scheduling Loop. Each cuPHY instance writes the NeuralRx result of its TBs to a decision page (❶). At the NeuralRx deadline, Backstop applies all results at once to the pending-recovery table

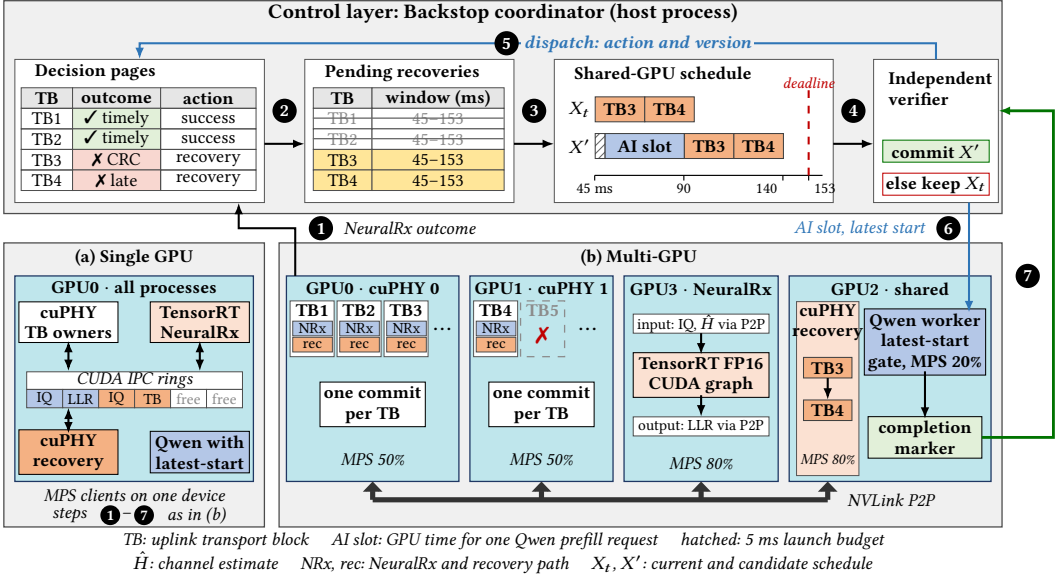


Fig. 5. Backstop architecture for one period. Steps: NeuralRx result (1), all results applied at the NeuralRx deadline (2), recoveries rescheduled around an AI slot (3), independent check (4), actions sent (5), AI slot sent (6), and completion marker (7). The steps are drawn for the evaluated four-GPU node (b); on one GPU (a), the same processes exchange IQ, LLR, and TB buffers through CUDA IPC rings and follow the same steps. Each TB comes from a different cell; cuPHY instance 0 owns TB1-TB3, instance 1 owns TB4 and TB5, and ellipses mark further TBs.

(2). A fast scheduler proposes a candidate that reschedules the needed recoveries around one AI slot (3), and the independent verifier checks the worst-case recovery schedule, the buffer slots, and the version before it commits X' or keeps X_t (4). The coordinator then sends each action with its version (5) and sends the AI slot with a latest start time to the Qwen worker (6). A completion marker or a “not launched” reply frees the slot (7), and the shared GPU runs the needed recoveries in the committed order.

Scheduling Inputs. The scheduler uses only information that is visible at the current event. A radio job and an AI job are

$$J_i = (r_i, d_i, h_i, c_i, K_i, q_i), \quad A_a = (u_a, v_a, \ell_a, b_a, g_a), \quad (1)$$

where r_i and d_i are the arrival time and the result deadline, h_i is the owning cuPHY instance, c_i is the time bound of the conventional recovery, K_i holds the NeuralRx workers that may decode it, and q_i is the value of trying NeuralRx; u_a, v_a, ℓ_a, b_a , and g_a are the AI arrival time, value, deadline, time bound, and margin. Decisions cannot use future CRC results, future arrivals, actual service times, or hidden channel labels.

Validated Modes. A time bound can be used for scheduling only under the measurement conditions in which it was validated,

$$\pi = (\text{GPU, placement, MPS, lifecycle, co-run, software}). \quad (2)$$

Therefore, a bound measured on a warm local path does not hold for a cold, restarted, or remote path. The scheduler marks a state UNVALIDATED whenever a needed bound was not measured under the current conditions, even if a sample finishes in time.

Proposer and Verifier. The fast scheduler may use a simple list heuristic and may miss a feasible placement. It cannot create an unsafe execution because only the verifier can commit a candidate. With this split, the evaluation can compare decision quality with an exact search while safety does not depend on the heuristic.

3.2 Pending-recovery scheduling

A TB whose NeuralRx result is unknown may still need a recovery, even though no conventional kernel is queued. Backstop records this work as a pending recovery

$$R_i = (a_i(t), d_i - g_i, c_i, h_i), \quad (3)$$

where $a_i(t)$ is its earliest allowed start. For the set $U(t)$ of TBs without a result, an *all-fail schedule* gives every R_i its own interval $[s_i, s_i + c_i]$ on its recovery GPU, after its arrival and before $d_i - g_i$, as if every one of these TBs failed. Data copies and AI slots that use the same GPU belong to the same schedule.

All-Fail Dominance. Backstop needs one schedule, not one schedule per combination of results.

PROPOSITION 1 (ALL-FAIL DOMINANCE). *If a timely NeuralRx success only removes its conventional recovery and does not add work to any other job, a feasible all-fail schedule for $U(t)$ is feasible for every subset $F \subseteq U(t)$ of failed TBs.*

PROOF. Delete the successful TBs from the all-fail schedule. The arrival, deadline, capacity, and non-overlap constraints of every remaining interval stay the same. $\square$

This result replaces $2^{|U|}$ possible schedules with one executable schedule. It does not apply when a success creates new mandatory work or changes another time bound; that case needs a new mode.

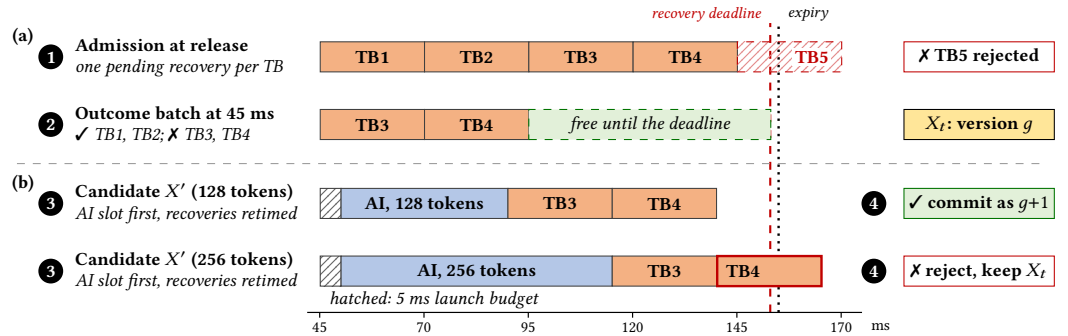


Fig. 6. One period on the shared GPU. (a) When TBs arrive, Backstop reserves one recovery per TB for the case that all of them fail and rejects TB5; at 45 ms, it applies both successes at once and cancels two pending recoveries. (b) The verifier commits a 128-token AI slot and keeps X_t for a 256-token slot, whose last recovery would end after the recovery deadline. AI requests are Qwen prefill requests; hatching marks the 5 ms launch budget.

Admission and Results. Figure 6a shows one period on the shared GPU. When TBs arrive, Backstop reserves one pending recovery per TB before it starts any NeuralRx (1). TB1-TB4 fit

before the recovery deadline, and TB5 is rejected because its reservation would cross the deadline. Therefore, a TB is never admitted without a conventional path. At the 45 ms NeuralRx deadline, Backstop applies all results at once (②): TB1 and TB2 succeed and cancel their pending recoveries, while TB3 and TB4 fail and keep theirs. The GPU time that TB1 and TB2 free stays available until the recovery deadline, and the committed schedule X_t holds two recoveries. Applying the same results one by one is wrong because it can create intermediate states that never existed together.

Candidate AI Slot. Figure 6b adds one AI slot to X_t . For a 128-token request, the candidate X' places the 5 ms launch budget and the AI slot first and moves TB3 and TB4 behind them (③). The last recovery still ends before the recovery deadline, and the verifier commits X' as version $g + 1$ (④). For a 256-token request, TB4 would end after the recovery deadline; the verifier rejects X' and keeps X_t .

Early AI Slot. The shared GPU is idle from the TB arrival until the NeuralRx deadline, when the first recovery can start. Therefore, Backstop also runs the same check when TBs arrive: an *early AI slot* starts right away, and the verifier commits it only if the all-fail schedule of every admitted TB, with the slot added, still ends before the recovery deadline. In our test timeline, four recoveries take 100 ms of the 108 ms recovery window, and an early slot may end at most 8 ms after the NeuralRx deadline. This limit admits a 16-128-token request and rejects a 256- or 512-token request. On one GPU, the early slot shares the GPU with NeuralRx. If the slot delays an NeuralRx result past the NeuralRx deadline, that TB only needs its recovery, which the all-fail schedule already holds. Thus, an early slot can cost NeuralRx value but not radio safety.

One Shared State. The schedule covers everything that shares a GPU: GPUs that share nothing are scheduled separately, while cuPHY instances that share a recovery GPU need one common set of pending recoveries. The full runtime state is

$$X_t = (S_t, Q_t, T_t, L_t, E_t, C_t, G_t), \quad (4)$$

where S is the recovery schedule, Q and T are the buffer slots of the receivers and of the data copies, L holds the AI slots, E holds the completion events and worker epochs, C is the radio commit state, and G records which cuPHY instance may launch AI. The verifier commits a private candidate X' only when

$$\begin{aligned} \text{VALID}(X') \equiv & \text{ExecutableRecovery}(S') \wedge \text{UniqueCredits}(Q', T') \\ & \wedge \text{TimelyAI}(L', E') \wedge \text{CurrentVersions}(X'). \end{aligned} \quad (5)$$

A rejected candidate leaves X_t unchanged, which rules out partial updates and orphan AI slots.

Safety Invariant. The verifier and the enforcement in Section 3.3 keep one invariant at every event.

SAFETY INVARIANT 1 (RECOVERY, OWNERSHIP, AND SINGLE COMMIT). *For every admitted TB and every allowed combination of pending results, a conventional path that overlaps no other work finishes before the result deadline. No receiver, copy, AI, or output buffer slot is reused before the GPU reports that its work finished, and each TB commits at most one result.*

3.3 Enforcing the schedule on the GPU

A correct schedule on the host is not enough unless launches and ownership follow it. Figure 7a follows one committed AI slot L across the coordinator, the marker files, the Qwen worker, and the GPU stream, and Figures 7b-e show each injected fault on the same kind of timeline.

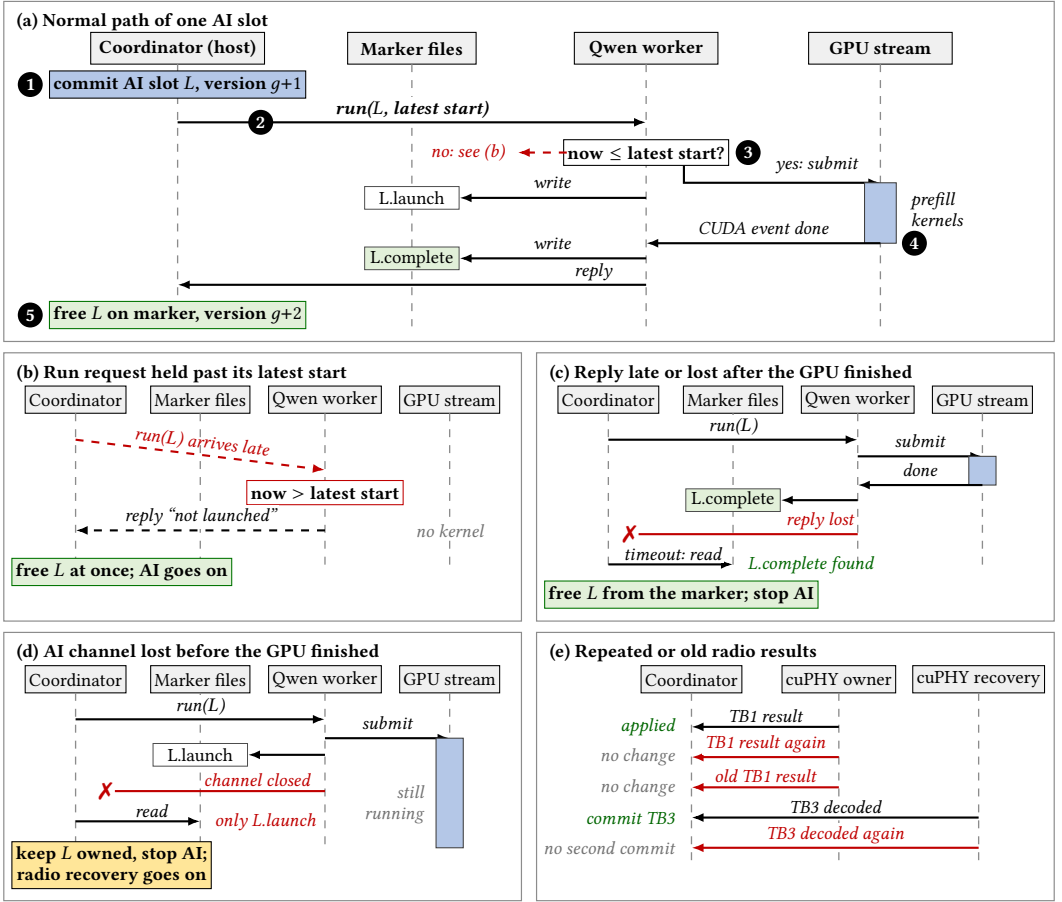


Fig. 7. Enforcing one AI slot on the GPU. (a) Normal path: commit with a latest start time (1), run request (2), launch check and launch marker (3), GPU completion and completion marker (4), and freeing on a matching marker (5). (b)-(e) Injected faults on the same kind of timeline: (b) a run request that arrives after its latest start time, (c) a reply that is late or lost after the GPU finished, (d) an AI channel lost before the GPU finished, and (e) repeated or old NeuralRx results (TB1) and recovery responses (TB3). Red marks the fault, and the box at the bottom of (b)-(d) shows what the coordinator does.

Launch Check. The coordinator commits L as version $g + 1$ (1) and sends $run(L, latest\ start)$ to the Qwen worker (2). The request carries the schedule version, the worker epoch, the buffer slots, and the latest start time. The worker launches only if the current time is at or before the latest start time (3); it then submits the prefill kernels and writes the launch marker $L.launch$. Otherwise, it replies "not launched" and runs nothing, and the coordinator frees L at once (Figure 7b).

Freeing GPU Time. When the GPU finishes, the worker writes the completion marker $L.complete$ and replies (4). The coordinator frees L only on a marker that matches its version and moves to version $g + 2$ (5). An RPC reply does not prove that the GPU finished. Therefore, slots and buffers stay owned until the matching GPU completion event, and the end of a data copy does not count as the end of a receiver or a recovery.

Fault Handling. Figures 7c-e show the other injected faults. When the reply is late or lost after the GPU finished (c), the coordinator times out, finds L .complete, frees L from the marker, and stops AI admission. When the channel is lost before the GPU finished (d), only L .launch exists; the coordinator keeps L owned because its kernels may still run, stops AI admission, and continues radio recovery. A repeated or old NeuralRx result leaves the radio state unchanged, and a repeated recovery response produces no second commit because each TB commits once (e).

Buffer Ownership. One AI queue can serve several cuPHY instances, but each instance has at most one pending launch request at a time, and a commit hands over the right to launch only while the local and common schedules are current. CUDA IPC and NVLink avoid a copy through host memory and also make ownership explicit: each buffer slot has one version, one completion state, and one completion event that the scheduler sees. Therefore, Backstop counts data transfer inside the validated service time. Reconnecting the same worker keeps the version through the markers, while replacing a worker process without a durable log remains UNVALIDATED.

3.4 Scheduling algorithm

Backstop runs one algorithm at every event, and only the verifier can commit its result. Algorithm 1 rejects optional work whenever it cannot show that the measurement conditions hold, that every recovery fits, or who owns each buffer.

Algorithm 1: SCHEDULEEVENT($X_t, \mathcal{E}_t, \mathcal{A}_t, \pi$)

- (1) If a needed bound was not validated under π , return UNVALIDATED and stop new optional work.
- (2) Apply all events $\mathcal{E}_t$ at once: add a pending recovery for each new TB, cancel it only for a timely success, and keep it for a failure, a late result, or a missing result.
- (3) Place every needed recovery on its recovery GPU. If no all-fail schedule exists, return INFEASIBLE and reject new NeuralRx and AI admission.
- (4) Visit the waiting AI requests in a fixed order. For each request, build X' by moving the recoveries and adding one AI slot.
- (5) If the verifier accepts X' , commit it, send its first action, and return SAFE+AI. Otherwise discard X' and keep X_t .
- (6) If no AI request fits, commit the recovery-only schedule and return SAFE-ONLY. The next completion event ends ownership and runs SCHEDULEEVENT again.

Fig. 8. Backstop scheduling algorithm. The visiting order decides efficiency, and the verifier decides safety. Thus, the heuristic can reject a safe candidate but cannot admit an unsafe one.

Event Processing. Step 1 stops new optional work and returns UNVALIDATED when a needed bound was not validated under the current conditions π . Step 2 applies all events at once: it adds a pending recovery for each new TB, cancels it only for a timely success, and keeps it for a failure, a late result, or a missing result. Step 3 places every needed recovery and returns INFEASIBLE when no all-fail schedule exists. Steps 4 and 5 visit the waiting AI requests in a fixed order, build one candidate per request, and commit the first candidate that the verifier accepts, which returns SAFE+AI. Step 6 commits the recovery-only schedule and returns SAFE-ONLY when no AI request fits.

When AI May Go First. For m_h recoveries of length c_h that must finish by the result deadline d_h minus the margin g_h , let b_{eff} include the launch overhead, the AI service time, and the completion margin. Running AI before the recoveries is safe under the following sufficient condition:

$$T_{\text{dec},h} + b_{\text{eff}} \leq d_h - g_h - m_h c_h. \quad (6)$$

Table 3. The four states of a scheduling decision.

State	Meaning
SAFE+AI	The worst-case recovery schedule fits, and at least one validated AI request fits too.
SAFE-ONLY	The worst-case recovery schedule fits, but no waiting AI request fits safely.
INFEASIBLE	Even the recoveries alone do not fit under the known bounds.
UNVALIDATED	A needed bound was not measured under the current conditions or failed validation.

The right side is the latest time at which the first needed recovery can start. Therefore, the condition counts the decision and launch delay and does not assume that the AI slot starts immediately.

Idle Time. The same condition shows why idle GPU time alone cannot justify admitting AI.

PROPOSITION 2 (IDLE TIME IS INSUFFICIENT). *On one GPU that runs one job at a time, with common recovery deadline D , pending recovery bounds $c_1, \dots, c_m$, and AI bound b_{eff} , suppose that the recoveries alone fit but*

$$t + b_{\text{eff}} + \sum_{i=1}^m c_i > D. \quad (7)$$

Then admitting AI into idle time while ignoring pending recovery cannot guarantee the radio deadline.

PROOF. Take the allowed case in which AI and every recovery take their full bounds. The last recovery then ends after D by Equation 7; GPU time that is idle now does not remove recovery work that may come later. $\square$

How Much AI Fits. The same reasoning predicts how much AI each policy can admit. Let W be the window up to the recovery deadline, L the AI deadline, a one complete AI slot, c one recovery bound, M_g the pending recoveries that static reservation holds, and F_g the needed recoveries on recovery GPU g . With $[x]_+ = \max(0, x)$,

$$K_{\text{static}} = \sum_g \left\lfloor \frac{[\min(L, W - M_g c)]_+}{a} \right\rfloor, \quad (8)$$

$$K_{\text{RF}} = \sum_g \left\lfloor \frac{[\min(L, W) - F_g c]_+}{a} \right\rfloor, \quad (9)$$

$$K_{\text{Backstop}} = \sum_g \left\lfloor \frac{[\min(L, W - F_g c)]_+}{a} \right\rfloor. \quad (10)$$

Static reservation holds every pending recovery, recovery-first runs the needed recoveries before AI, and Backstop may place AI first as long as $F_g c$ of recovery time still fits before the recovery deadline. For mixed BurstGPT request sizes, the model enumerates every subset, GPU, order, and recovery position exactly. Failure probabilities predict the expected AI value, but the all-fail safety check does not depend on them.

Decision States. Table 3 lists the four states of a decision. They keep a scheduling limit apart from missing evidence: SAFE+AI and SAFE-ONLY keep the same radio invariant, INFEASIBLE shows overload under the known bounds, and UNVALIDATED refuses to reuse evidence from a different setup.

3.5 What Backstop guarantees

Backstop gives a conditional guarantee for an explicitly validated mode. Validation ties every time bound to the setup in which it was measured and defines which events the runtime must handle.

Mode Conditions. A mode fixes the GPU topology, MPS settings, process lifecycle, worker epoch, memory residency, co-running AI, software, and the measured end-to-end time bounds. Changing any of them requires revalidation. The 180 ms period and the result deadline in our evaluation are test settings, not 5G slot or production HARQ timing.

Handled Events. The runtime handles NeuralRx failure and lateness, stale and duplicate messages, a lost reply before or after a control action, a missing AI reply after the GPU finished, and reconnection of the same worker. It stops new AI admission when completion evidence is missing.

Excluded Events. Arbitrary GPU hangs, driver resets, unbounded host stalls, clock errors in production, and workloads without validated bounds remain outside the claim. If the mandatory radio load alone does not fit, the state is INFEASIBLE; Backstop does not weaken the safety check to admit it.

3.6 Backstop implementation

Backstop maps the model onto long-running CUDA workers and explicit control objects, which makes the schedule state visible and enforceable across process and GPU boundaries.

Radio and AI Workers. MIG is disabled and MPS is enabled. cuPHY instances, NeuralRx workers, conventional recovery, and external AI run in separate long-running workers. TensorRT NeuralRx workers keep preallocated buffers and CUDA graphs, cuPHY workers run conventional recovery, and a Qwen2.5-1.5B worker runs prefill requests of fixed sizes. On the four-GPU node (Figure 5b), GPU0 and GPU1 host the cuPHY instances with a 50% MPS share, GPU3 hosts the NeuralRx worker, and GPU2 hosts the cuPHY recovery worker and the Qwen worker, which runs as a low-priority MPS client with a 20% share. On one GPU (Figure 5a), the same workers share the GPU with the same MPS shares. One cuPHY receiver of our test setup holds 8.6 GiB, and the recovery worker runs one recovery at a time. Therefore, on one GPU the recovery worker keeps one receiver for all cells.

Data Transfer and Control. CUDA IPC rings connect processes on one GPU, and NVLink rings connect GPUs. Each buffer slot carries a request version and a completion state. The end of a copy does not mean that a receiver or a recovery finished; the owning worker publishes a separate completion event. The coordinator stores the recovery schedules, the buffer slots, the worker epochs, the AI launch rights, and the commit state. Unix-domain RPC carries control messages, while GPU data stays on the IPC and NVLink paths.

Schedule Builder and Verifier. Backstop separates building a schedule from approving it. The fast scheduler visits the TBs and the waiting AI requests in fixed orders and outputs a complete schedule, the ownership changes, and the mode fingerprint; exact search serves only as a reference on small states. The independent verifier checks the job identity, arrival, validated service time, result deadline, GPU capacity, overlap, AI slots, buffer slot uniqueness, and version, and the runtime cannot run a candidate that the verifier rejects. The executor runs the first action of an approved schedule, waits for its completion event, and plans again; it never reorders GPU work from an unchecked host queue.

Measurement. Our instrumentation separates decisions, host progress, GPU execution, and GPU completion, which keeps a host timestamp from being reported as a kernel boundary. The runtime records arrival, admission, dispatch, worker observation, CUDA completion, commit, request and payload identity, buffer version, worker epoch, and the time a slot is freed, and Nsight traces check GPU ordering for selected modes. The validated path uses pyAerial with long-running workers.

A later native C++ path checks only input layout and decoder output against the Python path; it is not validated for timing, and its measurements are never mixed with the time bounds of the Python mode. Manifests keep source hashes, protocols, seeds, excluded nodes, analyzer versions, failures, and result hashes, and every result in this paper maps to a machine-readable file and field.

4 Evaluation

4.1 Evaluation setup

Hardware. We evaluate Backstop on two independent nodes, each with four NVIDIA A100-SXM4-40GB GPUs connected by third-generation NVLink. The bursty-arrival, load, and single-GPU campaigns (Sections 4.5 and 4.6) run on further nodes of the same system with four A100-SXM4-80GB GPUs each; a single-GPU run uses one of the four GPUs. MIG is disabled and MPS is enabled on every GPU. On the four-GPU node, GPU0 and GPU1 run the two cuPHY instances with a 50% MPS share, GPU3 runs the TensorRT NeuralRx with 80%, and GPU2, the shared GPU, runs the cuPHY recovery worker with 80% and the Qwen worker with 20% and low priority. We withhold the facility name for anonymous review.

What We Measure. Sections 4.2 and 4.3 use a capacity model checked by exact search and BurstGPT traces, without running GPU kernels, to measure how much AI fits safely and how much of it is served on time. Sections 4.4-4.9 run the complete scheduler on physical GPUs and measure online scheduling, AI service under bursty arrivals, the single-GPU placement with early AI slots, whether rejected requests are really unsafe, the runtime safeguards under injected faults, prediction accuracy, and decision cost. Section 4.10 covers the MPS setting and the worker lifecycle, and Appendix A varies the recovery bound.

Workloads and Test Timeline. Radio work uses NVIDIA Aerial cuPHY [14] and a TensorRT FP16 NeuralRx. The evaluated warm mode uses a 180 ms period and a 155 ms result deadline (P180/D155). Its validated end-to-end time bounds are $B_{\text{NRx}} = 45$ ms for NeuralRx, $B_{\text{conv}} = 25$ ms for a recovery, and $B_{\text{AI}}(\text{context64}) = 35$ ms for a 64-token AI request, with a 2 ms margin before the result deadline. The period is not a 5G slot, and the result deadline is not a production HARQ deadline. In every period, four admitted cells each send one single-UE, 273-PRB uplink TB; the cuPHY instance on GPU0 serves three of them and the instance on GPU1 serves one. Qwen2.5-1.5B prefill [17] with 16-512-token prompts adds AI load on the GPU, and BurstGPT [24] supplies arrival times and prompt sizes from four separate windows of its public trace and the prompts of the bursty campaign. Synthetic channels drive the timing experiments; a separate Sionna CDL-D/E mode tests the radio value of NeuralRx (Section 6).

Baselines. Every baseline makes the same radio decisions and sees the same arrivals, bounds, and workers as Backstop. Two baselines follow the sharing approaches of Section 5. *Static reservation* holds the recovery of every admitted TB until the result deadline, as reservation-based sharing does, and *idle-time admission* admits AI into GPU time that is idle when it decides and ignores pending recovery, as sharing that uses any idle GPU time does. The *recovery-first* policy is an ablation of Backstop: it keeps pending-recovery tracking, the all-fail admission check, and one commit per TB, but it runs every needed recovery before AI, which isolates the effect of putting AI first. The *best possible* schedule sees the whole trace in advance and bounds the value that any policy can reach. Idle-time admission and uncontrolled MPS serve only to expose wrong assumptions; they are not safe alternatives.

How We Run Experiments. Before looking at results, each main campaign fixes its source code, workload, bounds, seeds, nodes, excluded nodes, analyzer, and stopping rule. Development and held-out runs use different seeds and nodes, and failed, negative, and UNVALIDATED runs stay in

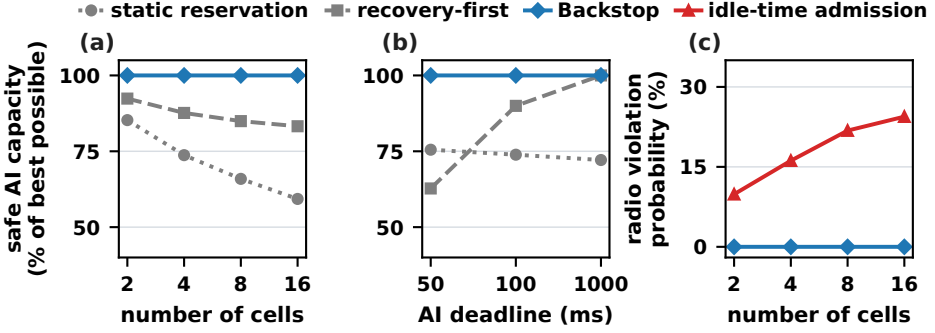


Fig. 9. Model results without GPU runs: safe AI capacity in the capacity model, checked by exact search over the 13,716 settings in which the recoveries alone fit. (a) AI capacity relative to the best possible schedule as the number of cells grows. (b) AI capacity as the AI deadline tightens. (c) Probability that idle-time admission breaks the radio deadline; static reservation, recovery-first, and Backstop never break the all-fail schedule. Idle-time admission is left out of (a) and (b) because it admits more AI than is safe.

the record. We report zero observed events as results on a finite sample, not as probabilities or worst-case execution time (WCET) guarantees.

Reproducibility. The anonymous artifact package contains the source code, protocols, manifests, analyzers, exact checker, independent verifier, result summaries, and figure scripts. The GPU campaigns need proprietary Aerial/cuPHY and TensorRT components; the model, checker, trace, and analysis do not. Node names and absolute paths are removed from the review artifact.

Use of generative AI. Generative-AI tools helped with language editing, organization, bibliography cleanup, and consistency-checking scripts. The authors designed the system and model, fixed the protocols, ran the experiments, inspected failures, and checked every claim against hashed artifacts. Generative AI produced no experimental measurement or ground-truth label.

4.2 How much AI fits

Capacity across Cells and Deadlines. Figure 9 evaluates the capacity model over 21,384 settings that vary the number of cells (one TB per cell and period), cuPHY instances, GPUs, NeuralRx success probability and correlation, result deadline, recovery bound, AI request size, AI deadline, and offered load. At 7,668 settings, the recoveries alone do not fit. The model decides all AI requests at one time after the NeuralRx deadline, and at all 13,716 remaining settings Backstop admits as much AI as the best possible schedule. Static reservation reaches 85.3% of the best with two cells and 59.3% with 16 cells because it holds every pending recovery until the result deadline. Recovery-first reaches 92.4% and 83.3% because it runs the needed recoveries before AI; with a 50 ms AI deadline, it reaches only 62.7%. Backstop admits more AI than static reservation at 3,600 settings (26.2%), and recovery-first stays at least 5% below the best at 2,604 settings (19.0%). Idle-time admission admits more AI than the best possible schedule, but its probability of breaking the radio deadline rises from 9.9% with two cells to 24.4% with 16 cells and is positive at 4,338 settings (31.6%). Checking every order exhaustively agrees with the model in all 5,376 checked cases. Backstop reaches this capacity because it admits AI against the all-fail schedule of the remaining recoveries and puts AI first whenever that schedule still ends before the recovery deadline.

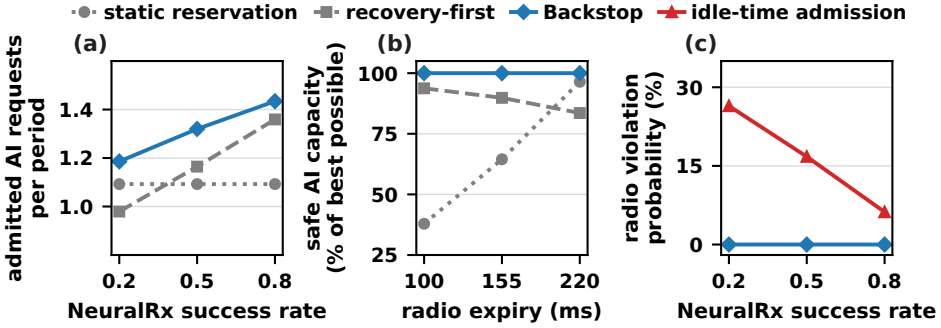


Fig. 10. Model results: what drives safe AI capacity. (a) Mean AI requests per period against the NeuralRx success rate. (b) AI capacity relative to the best possible schedule against the result deadline. (c) Probability that idle-time admission breaks the radio deadline against the NeuralRx success rate. Each success rate covers the same 4,572 settings in which the recoveries alone fit.

Where the Throughput Comes From. Tracking pending recovery gives most of the throughput gain. Static reservation holds every pending recovery until the result deadline and reaches only 59.3–85.3% of the AI capacity of Backstop across cell counts. Recovery-first tracks pending recovery like Backstop and reaches 83.3–92.4%, because both policies fit the same AI work and needed recoveries into one window. The rest of the gap comes from order: recovery-first loses the AI requests whose deadline passes while the recoveries run first. Therefore, with a 1,000 ms AI deadline, recovery-first reaches 100% of the capacity of Backstop, and with a 50 ms deadline it reaches 62.7% (Figure 9b).

Capacity Freed by NeuralRx Success. Figure 10a isolates the GPU time that a timely NeuralRx success frees. Each success rate covers the same 4,572 settings. As the success rate grows from 0.2 to 0.8, Backstop admits 1.19, 1.32, and 1.43 AI requests per period on average, while static reservation stays at 1.09 because it holds every pending recovery whatever the result. Recovery-first rises from 0.98 to 1.36 requests and admits fewer requests than static reservation at a 0.2 success rate, because it runs the many needed recoveries before any AI request. Backstop turns each success into AI capacity because the success cancels the pending recovery of its TB at the NeuralRx deadline and Backstop puts AI first against the remaining all-fail schedule.

Result Deadline. Figure 10b varies the result deadline. At 100 ms, static reservation reaches only 37.9% of the best because the reserved recoveries fill most of the short window; at 220 ms, it reaches 96.5%. Recovery-first moves the other way, from 93.8% to 83.6%, because a longer window makes the AI deadline the limit, and running the recoveries first then delays AI past its deadline. Backstop stays at the best capacity at every result deadline. Figure 10c shows the cost of ignoring pending recovery: the probability that idle-time admission breaks the radio deadline falls from 26.4% to 6.2% as the success rate grows, and it is highest when NeuralRx fails most often.

4.3 AI served on real traces

Where Order Matters. A screen fixed in advance maps four BurstGPT windows onto 2,592 settings of topology, failures, recovery, AI SLO, and offered load, of which 2,052 fit the needed recoveries. With a 50 ms AI SLO, even the best schedule places no AI request. With 100, 250, and 1,000 ms, recovery-first reaches 75.8%, 98.0%, and 100% of the best on average. The screen keeps the 362 settings at which recovery-first falls at least 5% behind the best in every window: 298 at a 100 ms SLO and 64 at 250 ms.

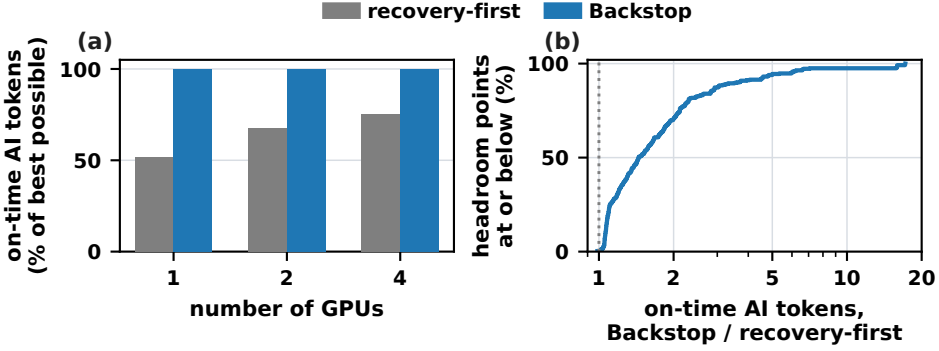


Fig. 11. AI tokens served within their SLO on BurstGPT traces, at the 362 settings chosen in advance because recovery-first falls at least 5% behind the best placement there. (a) Mean value relative to the best fixed placement for each GPU count. (b) Cumulative distribution of the ratio between the value of Backstop and that of recovery-first.

Value at These Settings. Figure 11a shows that Backstop closes this gap. On one GPU, recovery-first delivers 51.7% of the best fixed placement and Backstop delivers 99.8%; on two and four GPUs, recovery-first delivers 67.4% and 75.5%, while Backstop stays above 99.8%. Figure 11b shows that Backstop delivers 1.44 $\times$ the value of recovery-first at the median and up to 17.2 $\times$. The AI-first greedy order of Backstop is within 5% of the best fixed placement at 361 of the 362 settings, and the remaining setting has a 7.02% gap. Backstop gets there by placing AI before the needed recoveries whenever the all-fail schedule still ends before the recovery deadline, while recovery-first puts every AI request behind the recoveries and misses tight AI SLOs. Moving recoveries flexibly across GPUs would help at 203 settings in an upper-bound model: 0/102 with one GPU, 145/175 with two, and 58/85 with four, with a 10.64% median gain. Because this model ignores the cost of moving work and was not validated on GPUs, it is a design target, and we make no executed optimizer claim. Choosing among the fixed Sionna channel settings to reduce pending recoveries, with no or 0.02 loss of radio value, removes no pending recovery.

Original Trace. The gain depends on how much room the workload leaves. In the original physical mode, a calibration-only exact screen gives static reservation, recovery-first, Backstop, and the best schedule the same radio decisions, arrivals, bounds, and one Qwen request per period. After we fixed an error in how absolute deadlines were read, without dropping any request or opening the held-out data, exact matching assigns 856 timely requests and 349,387 tokens to recovery-first, Backstop, and the best schedule, and static reservation leaves no validated Qwen slot in the same period. Charging every needed recovery its full bound leaves recovery-first at 345,405 tokens, only 1.14% below Backstop and below the 5% threshold set in advance. Thus, the original trace leaves no room for ordering, and recovery-first stays the strong safe baseline for it.

4.4 Scheduling on physical GPUs

Two-Node Campaign. The fixed P180/D155 campaign runs 600 periods on each of two nodes and offers one Qwen request of 16-512 tokens in every period. It runs 4,800 TensorRT NeuralRx requests, 1,312 cuPHY recoveries on the shared GPU, 1,077 Qwen requests, and 4,800 radio commits, one per TB, without breaking any deadline, bound, buffer rule, or commit rule; 0/4,800 deadline misses gives a 95% upper bound of 0.0625% by the rule of three. The slowest NeuralRx, recovery, and radio-commit paths take 18.123, 13.465, and 118.022 ms against bounds of 45, 25, and 155 ms.

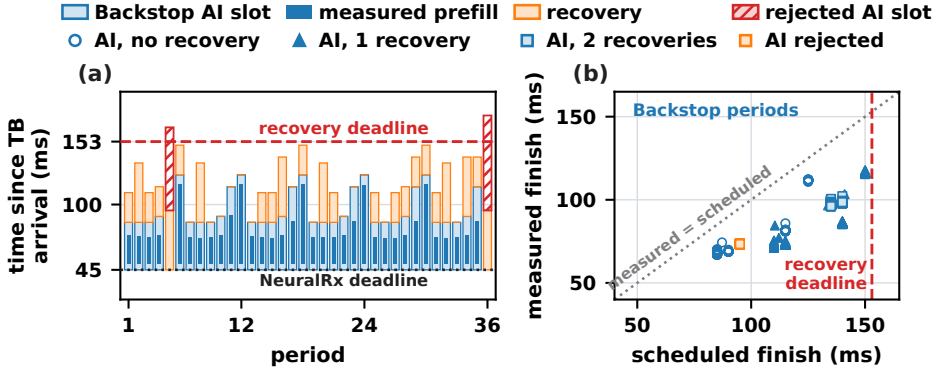


Fig. 12. Scheduling in the two-node campaign; time is measured from the TB arrival, as in Figure 1. (a) Committed schedule of the shared GPU in 36 consecutive periods: each admitted AI slot starts at the 45 ms NeuralRx deadline and includes the 5 ms launch budget, and the needed recoveries follow with their full bound; a rejected AI slot is drawn where it would have ended. (b) Measured against scheduled finish of the last action on the shared GPU, the AI prefill or the last recovery, in all 1,200 periods; the marker gives the number of needed recoveries, orange marks a rejected AI request, and the dotted diagonal marks equality.

Nsight shows 191 intervals, 8.932 ms in total, in which NeuralRx and Qwen kernels run at the same time, and conventional recovery overlapping the NeuralRx of another cell in 12/12 checked windows for 12.309 ms. This confirms that the work really runs concurrently; it is not a time bound. **AI-First Schedules.** Figure 12a shows the schedules that Backstop commits in 36 consecutive periods of the development run. In every period with AI, the AI slot takes the shared GPU first at the 45 ms NeuralRx deadline, and the needed recoveries follow with their full 25 ms bound and still end before the 153 ms recovery deadline. Backstop rejects an AI request only when its slot and the needed recoveries would cross the recovery deadline, as for the 256-token request of period 5 and the 512-token request of period 36, which each meet two needed recoveries. Static reservation holds all four pending recoveries, or 100 ms of the 108 ms window, and admits no AI request in this mode.

Execution inside the Schedule. Figure 12b compares, for every period, the measured end of the last action on the shared GPU with the end that its schedule allows. Every period ends before its scheduled end, and every scheduled end lies before the recovery deadline. The latest measured end is 72.9 ms after the NeuralRx deadline, 35.1 ms before the recovery deadline, and the smallest margin to a scheduled end is 12.7 ms. The margin exists because the schedule charges every AI request its full bound and every recovery its full bound, while Backstop frees a slot when the GPU finishes and plans the remaining recoveries from that time. In two periods, the coordinator applies the NeuralRx results 23.8 and 31.0 ms late; Backstop checks the slot from the actual decision time, and both periods end before their scheduled ends. Thus, the committed schedule runs as planned on physical GPUs, and it accounts for the decision delay that it observes.

4.5 AI service under bursty arrivals

Bursty Workload. AI requests arrive in bursts, and during a burst the scheduler decides how much AI each recovery window can take. Therefore, we replay one sequence of 329 BurstGPT prompts on the physical four-GPU path at 3 requests/s for 108 s. The prompts are drawn from all BurstGPT requests and rounded up to the six prompt lengths, and 47.4% of them round to 512 tokens. Six arrival patterns use this sequence and differ only in arrival times: evenly spaced arrivals,

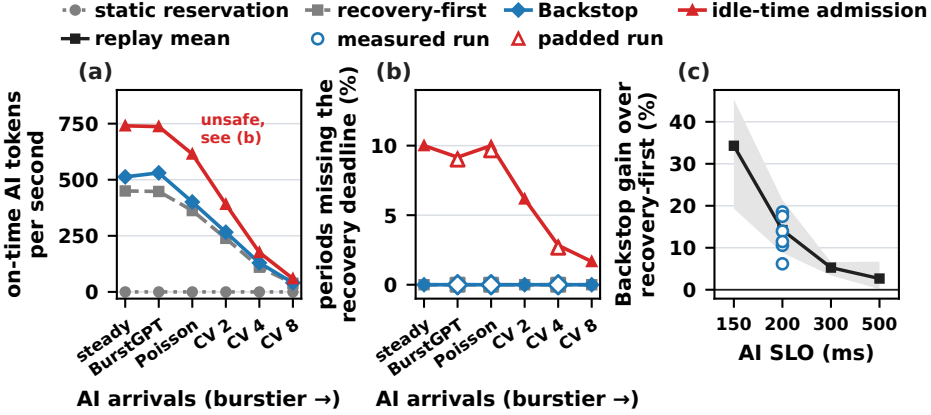


Fig. 13. AI service and radio safety under bursty AI arrivals on the physical four-GPU path. Every pattern carries the same 329 BurstGPT prompts at 3 requests/s for 108 s and differs only in arrival times; the AI SLO is 200 ms. (a) AI tokens served within their SLO; idle-time admission serves the most only because it breaks the recovery deadline in (b), and static reservation admits no AI request. (b) Periods in which the admitted AI breaks the recovery deadline at the declared bounds (lines, measured service times), and periods whose last recovery actually ends after the recovery deadline when every AI request and recovery takes its full bound (padded runs). (c) Gain of Backstop over recovery-first against the AI SLO: replay mean over the six patterns with its range (band), and the measured runs at 200 ms.

the per-second arrivals of a BurstGPT segment, and Gamma arrivals whose gaps have a coefficient of variation (CV) of 1 (Poisson), 2, 4, and 8. Every request must receive its first token within 200 ms, and up to four AI slots share one period in arrival order. All policies see the same NeuralRx results for a pattern, which makes each comparison paired.

AI Service. Figure 13a shows the AI tokens served within their SLO. Backstop serves 6.2-18.5% more on-time tokens than recovery-first under every pattern: 18.5% for BurstGPT arrivals, 10.6% for Poisson arrivals, and 6.2% for CV-8 bursts. Static reservation holds all four recoveries and serves no AI request in this mode. Idle-time admission serves 1.36-1.53 $\times$ the tokens of Backstop because it also admits requests whose recoveries no longer fit before the recovery deadline. As the arrivals become burstier, every policy serves fewer tokens because a burst is larger than one period can hold and its requests expire.

Radio Safety under Bursts. Figure 13b shows the price of that extra AI. Idle-time admission breaks the recovery deadline at the declared bounds in 1.7-10.0% of periods, most often with evenly spaced arrivals, where a request waits in almost every period. When every AI request and recovery takes its full bound, its last recovery actually ends after the recovery deadline in 2.7-9.7% of periods, and 128 radio results miss the result deadline. Backstop and recovery-first never miss the recovery deadline, and no radio result misses the result deadline in any of their 18 runs. In the same padded runs, Backstop serves 10.6-18.5% more on-time tokens than recovery-first.

Throughput under Load. Figure 14 raises the AI load with Poisson arrivals. With a 200 ms SLO, Backstop serves 10.6-21.2% more on-time AI tokens than recovery-first from 1.92 to 4.95 requests/s, and the two nearly tie at 0.98 requests/s. Static reservation serves no AI request at any load, and idle-time admission serves the most but breaks the recovery deadline at the declared bounds in 19-88 of 600 periods. With a 1,000 ms SLO (Figure 14b), Backstop and recovery-first reach the same throughput at every load because both fit the same AI work and needed recoveries into each

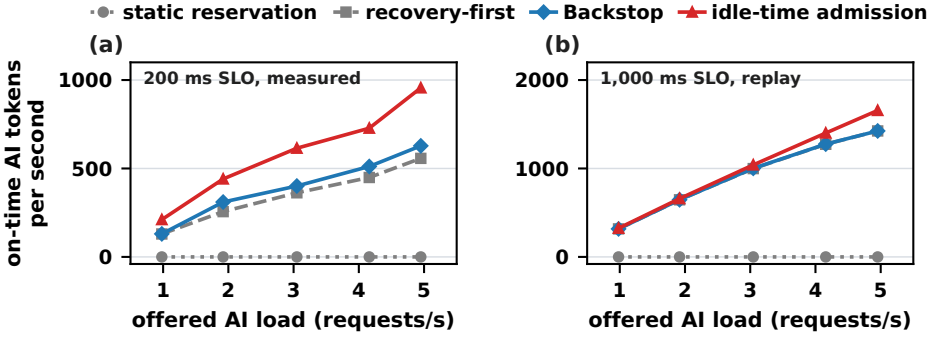


Fig. 14. On-time AI throughput as the load grows, with Poisson arrivals. (a) Measured runs with a 200 ms SLO; the 3.05 requests/s point is the Poisson run of Figure 13, taken on the other node. (b) Replay with the measured service times and a 1,000 ms SLO. Static reservation admits no AI request in this mode.

window. Thus, tracking pending recovery decides how much AI one window can hold, and putting AI first decides how much of it meets a tight deadline.

Where the Gain Comes From. Figure 13c varies the AI SLO in a replay that uses the measured service times of the two-node campaign, averaged over five seeds. The gain of Backstop over recovery-first shrinks as the SLO loosens: 34.3% on average at 150 ms, 14.2% at 200 ms, 5.3% at 300 ms, and 2.7% at 500 ms. The measured gains at 200 ms, 6.2-18.5%, lie close to the replay range of 8.8-21.4%. Backstop gets this gain because it places a checked AI slot ahead of the needed recoveries, while recovery-first must first give every needed recovery its full bound before an AI slot can meet its SLO. Thus, the order matters for interactive AI SLOs and fades for loose ones, which agrees with the tie on the original trace in Section 4.3.

4.6 Single-GPU placement

One-GPU Setup. Backstop also schedules a single GPU (Figure 5a). We rerun the bursty-arrival and load campaigns of Section 4.5 with every process on one A100-SXM4-80GB GPU: the four cuPHY owners with a 50% MPS share, the NeuralRx worker and the recovery worker with 80%, and the Qwen worker with 20% and low priority. Copies inside one GPU through CUDA IPC replace NVLink, and the radio protocol, seeds, bounds, traces, and policies stay the same. Thus, every single-GPU run pairs with its four-GPU run.

Memory on One GPU. Memory, not compute, limits the single-GPU placement first. One cuPHY receiver of our test setup holds 8.6 GiB, and the four-GPU placement peaks at 81.7 GiB summed over its GPUs, above the 80 GiB of one A100 (Figure 15c). With one receiver per cell for recovery, the single-GPU run runs out of memory. The recovery worker runs one recovery at a time, and a recovery depends only on the IQ window that the owner provides. Therefore, on one GPU the recovery worker keeps one receiver for all cells, and the GPU peaks at 54.3 GiB.

Same Decisions on One GPU. Figure 15a pairs the 30 natural runs on one GPU with their four-GPU runs. The on-time AI tokens match exactly in 27 pairs and differ by at most 3.6% in the other three. Under the bursty patterns, Backstop serves 6.2-18.5% more on-time tokens than recovery-first on both placements, and idle-time admission breaks the recovery deadline at the declared bounds in 1.7-10.0% of periods on both. The placements agree because Backstop separates the GPU work in time: NeuralRx runs before the NeuralRx deadline, and AI slots and recoveries run after it. Over the paired natural runs, the p99 of every service path moves by at most 1.6 ms; for example, NeuralRx results arrive within 6.7 ms against 7.0 ms.

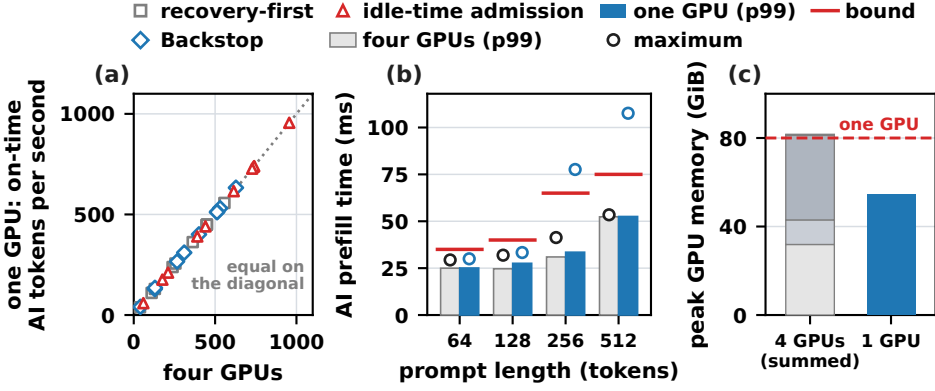


Fig. 15. One GPU against the paired four-GPU runs. (a) On-time AI tokens per second of the 30 paired natural runs (six arrival patterns at 3 requests/s and four Poisson loads); the diagonal marks equal service. (b) AI prefill time per prompt length over every run, including stopped and padded runs: p99 (bars) and maximum (circles) against the bound of each size (red). (c) Peak GPU memory of the four-GPU placement summed over its GPUs and of the single-GPU placement with one shared recovery receiver; the dashed line marks the memory of one A100.

Tails on One GPU. One GPU leaves the medians and p99 unchanged but makes rare slow runs longer (Figure 15b). Two of the 5,437 single-GPU prefills exceed their bound: a 256-token request takes 77.6 ms against 65 ms, and a 512-token request takes 107.6 ms against 75 ms. In the second case, the verifier finds, when it frees the slot, that the remaining recovery no longer fits before the recovery deadline, and the run stops. In addition, 5 of the 42 single-GPU runs lose an NeuralRx result at the first NeuralRx deadline after startup. In two of them, the owners stall for up to 0.35 s, the first period's recoveries miss the deadline, 8 radio results miss the result deadline, and the run stops. None of the 39 four-GPU runs shows any of these events. Thus, the single-GPU placement is a separate mode: it needs a warm-up before the first TB and time bounds measured on one GPU, and we report its runs as development evidence.

Early AI Slots. On one GPU, an early AI slot shares the GPU with NeuralRx (Section 3.2). We pair Backstop with and without early slots under the same traces, on one node except for the CV-8 pair. Figure 16a shows that early slots serve 2.6-70.3% more on-time AI tokens under the six arrival patterns, and the gain grows with burstiness: a burst leaves short requests waiting at the start of the next period, and an early slot serves them before the NeuralRx deadline. Under Poisson load, the gain is 5.4-14.5% (Figure 16c). Early slots do not delay NeuralRx: the p99 of the NeuralRx result stays within 6.5-7.2 ms, far below the 45 ms deadline, even in padded runs that hold every early slot on the GPU for its full bound (Figure 16b), and no run loses a timely NeuralRx success. No early-slot run misses the recovery deadline or the result deadline. Backstop gets this gain because the all-fail schedule already holds the recovery of every admitted TB when the TBs arrive. The planner skips slots that cannot end in time before it calls the verifier, which keeps planning within the 5 ms launch budget when a burst fills the queue.

4.7 Are rejected requests really unsafe?

What the Admission Check Prevents. Figure 17a runs three boundary cases, fixed in advance, on two independent nodes as a bound-padded diagnostic: every recovery and AI request takes its full declared bound. In the 256-token case (E4), two needed recoveries remain at 45 ms and a 256-token

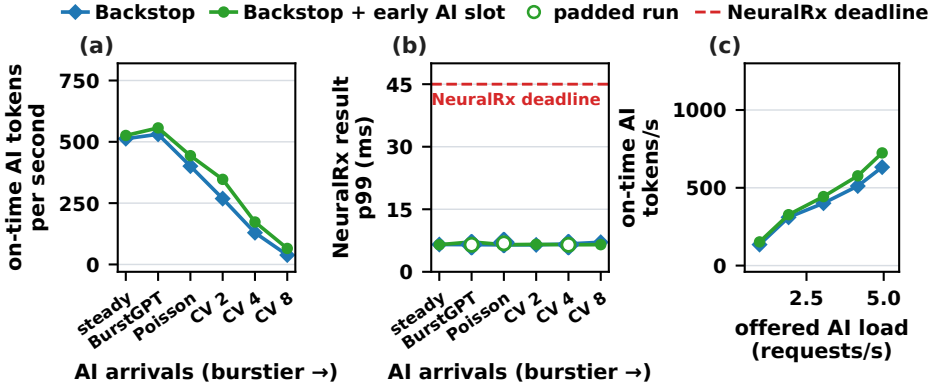


Fig. 16. Early AI slots on one GPU, paired with Backstop without them. (a) On-time AI tokens per second under the six arrival patterns at 3 requests/s. (b) p99 of the time from TB arrival to the NeuralRx result in natural runs (lines) and padded runs (circles), against the 45 ms NeuralRx deadline. (c) On-time AI tokens per second against the offered Poisson load, with a 200 ms AI SLO.

AI request arrives; idle-time admission launches it, and the last recovery ends 10.6-11.7 ms after the 153 ms recovery deadline in every attempt. In the 64-token case (E6b), one recovery remains and a 64-token request arrives at 89 ms; launching the request that Backstop rejects moves the last recovery 0.4-2.8 ms past the recovery deadline. Backstop rejects both requests, admits the same request at 88 ms (E6a), and ends every attempt before the recovery deadline. Across the selected rounds, idle-time admission in E4 misses both the recovery deadline and the result deadline in 40/40 rounds, the forced launch in E6b misses the recovery deadline in 40/40, and Backstop has 0/120 safety violations (95% upper bound 2.5%). The counts in which every job stayed within its bound are 39/39, 39/39, and 0/119, respectively.

Rejected Requests. Figure 17b shows the same boundary in the two-node campaign. Backstop runs every offered 16-128-token request and rejects 55 of the 256-token and 68 of the 512-token requests before launch. These are exactly the requests that would push the last of two needed recoveries past the recovery deadline. Idle-time admission would launch all 1,200 requests, including these 123, and static reservation would launch none. Thus, Backstop admits every request that the all-fail schedule allows and no request that it forbids.

4.8 Runtime safeguards and faults

Latest Start Time. A committed schedule stays valid only if the GPU cannot start a slot later than the schedule assumes. Figure 18a shows when each slot reaches the Qwen worker, relative to its latest start time. All 1,077 slots of the two-node campaign arrive 2.9-3.8 ms before the latest start time and launch. A fault that holds the AI slot delays 52 slots until 6.4-7.2 ms after the latest start time, and the worker refuses all 52 before launch with a “not launched” reply. Without this check, these requests would run up to 7.2 ms later than their schedules allow and push the needed recoveries toward the recovery deadline. Backstop prevents this by attaching the latest start time to the slot at commit and checking it in the worker right before kernel submission.

Radio Commit Time. Putting AI first delays the needed recoveries, and Figure 18b measures this cost on every TB of the two-node campaign. The 3,488 TBs with a timely NeuralRx success commit at the NeuralRx deadline, at a median of 46.1 ms after arrival, whatever the AI slot. Recovered TBs commit at a median of 62.1 ms when no AI slot runs before the recovery, 72.7-73.7 ms after

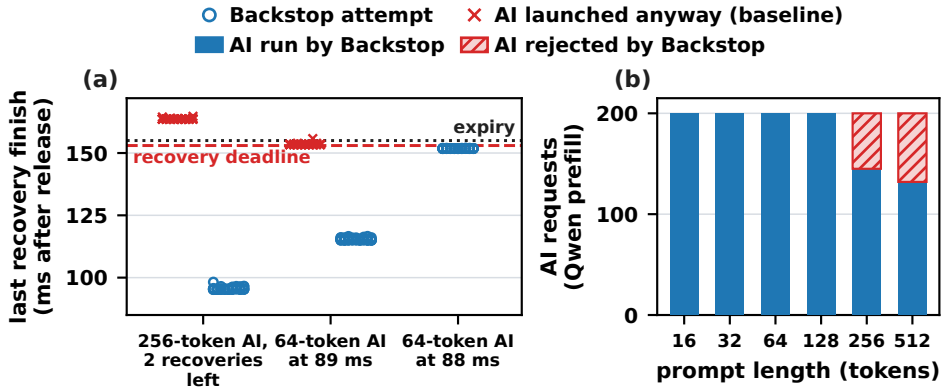


Fig. 17. Are rejected requests really unsafe? Two nodes. (a) End of the last recovery in every attempt of the bound-padded test. E4 offers a 256-token request with two needed recoveries at 45 ms; E6b and E6a offer a 64-token request with one needed recovery at 89 and 88 ms. The other policies launch the request anyway: idle-time admission in E4 and a forced launch of the request that Backstop rejects in E6b. (b) AI requests per prompt length in the two-node campaign: requests that Backstop runs and requests that it rejects because the needed recoveries would then miss the recovery deadline; idle-time admission would run all of them, and static reservation runs none.

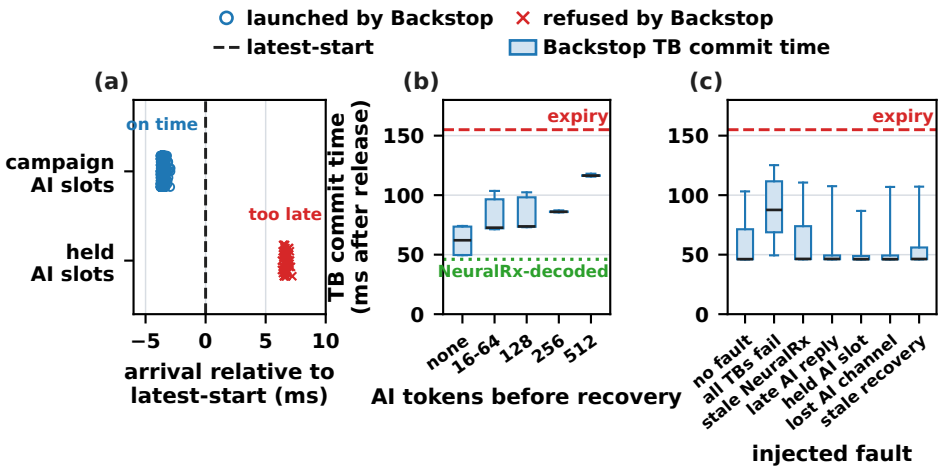


Fig. 18. Three checks of the runtime safeguards on physical GPUs. (a) Latest start time: arrival of each AI slot at the Qwen worker relative to its latest start time, for the 1,077 slots of the two-node campaign and the 52 slots that a fault holds past it. (b) Cost of putting AI first: commit time of every recovered TB, grouped by the prompt length of the AI slot that runs before its recovery; the dotted line marks the median commit of the TBs that NeuralRx decodes. (c) Faults: commit time of every TB under each injected fault: no fault, all TBs of a period fail, stale and duplicate NeuralRx results, a late AI reply after the GPU finished, an AI slot held past its latest start time, an AI channel lost before the GPU finished, and stale and duplicate recovery responses.

a 16-128-token slot, 86.0 ms after a 256-token slot, and 116.4 ms after a 512-token slot. The latest commit, 118.022 ms, is 37.0 ms before the 155 ms result deadline. Thus, putting AI first moves a

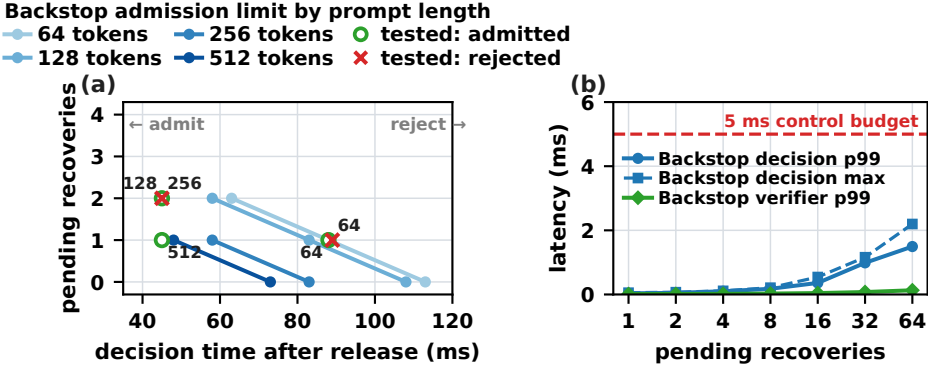


Fig. 19. Prediction and decision cost. (a) Latest decision time at which one AI request of each prompt length can still be admitted, with four admitted TBs and zero to four pending recoveries; circles and crosses mark the physical tests, fixed in advance, that the scheduler admitted and rejected, labeled with their prompt length. (b) Time to build and to check a candidate against the number of pending recoveries, with the 5 ms launch budget. The times are measured on finite runs, not WCETs.

recovery commit later by at most the scheduled slot length and never past the result deadline, because the schedule charges each slot before the recoveries that follow it.

Fault Handling. A stale message or a lost reply can free the wrong buffer slot even when the schedule is correct. Therefore, we combine model checking with fault injection on GPUs. A model checker explores 51 combinations of worker epoch, version, completion, and marker state and finds no invariant violation. The fault campaign injects seven kinds of faults on the two validated nodes. Figure 18c shows that every TB of every kind commits exactly once before the result deadline. The largest commit time is 125.2 ms, when every TB of a period fails and all four recoveries run, and it stays 29.8 ms before the 155 ms result deadline. The campaign runs 700 periods, 2,800 NeuralRx requests, 942 recoveries, and 2,800 commits without an observed deadline miss, including 40 replayed messages, four fatal faults, and 260 periods after them; 0/2,800 misses gives a 95% upper bound of 0.107%. On one additional node, the first NeuralRx result exceeded its bound, and we keep that run as UNVALIDATED without pooling it. Thus, fault handling holds on the two validated nodes for the declared fault and lifecycle kinds.

4.9 Prediction and decision cost

Prediction Agreement. Figure 19a shows the latest decision time at which each AI request size can still be admitted as pending recoveries pile up. The prediction matches exact search on 16,023 states and on all 1,200 past physical decisions. We then fix seven boundary cases, their order, and the seeds of each node. Across two independent nodes and 180 rounds, the model and the runtime agree in every round: a 64-token request at 88 ms is admitted in 30/30 rounds, while one at 89 ms or later is rejected in 30/30. The campaign runs 720 NeuralRx requests, 330 recoveries, 90 Qwen requests, and 720 commits without an observed deadline miss.

Decision Cost. On 600 small random states, exact search accepts 504 and the fast scheduler accepts 494, with zero unsafe acceptances and ten needless rejections; in the validated mode, the fast scheduler and exact search agree on all 16,023 states. Figure 19b scales the fast scheduler to 64 pending recoveries on 2,800 states with 1, 2, 4, and 8 parallel recovery workers. Every returned schedule passes the verifier. With 64 pending recoveries, the decision time is 1.494 ms at the p99 and 2.197 ms at the maximum, and the verifier takes 0.131 ms at the p99, within the 5 ms launch

budget. The cost stays low because the fast scheduler orders only the visible jobs and the verifier checks one proposed schedule.

4.10 MPS setting and worker lifecycle

MPS Setting. MPS controls how processes share the GPU but does not bound time. When an AI client overruns, the radio misses 2 of 1,500 deadlines with a 100% MPS cap and 1 of 1,500 with a 20% cap. In a matched lifecycle test, a CPU-only control misses 0 of 10,000 deadlines and a GPU client under MPS misses 12 of 10,000. The two tests ran separately and do not show that a smaller cap is always safer. Thus, neither the MPS cap nor the current queue can decide admission.

Worker Lifecycle. Restarting or idling a worker changes the service times that a schedule relies on. The lifecycle matrix contains five validated or partly validated subsets and five UNVALIDATED modes. The validated evidence covers warm long-running workers, the first request after 30 s idle, an MPS restart followed by revalidation, radio continuity while Qwen reloads, and reconnecting the same worker. Whether AI stays available during a restart or reload is outside the claim. Cold starts of a process or a model, 5 min and 30 min idle, garbage collection enabled, and replacing a worker process remain UNVALIDATED. A single-node replacement result is “Development canary only” because it lacks independent held-out runs. Durable journal recovery would be a new mechanism and remains outside this paper.

5 Related Work

GPU Sharing in AI-RAN and vRAN. Earlier work on AI-RAN and virtualized RAN (vRAN) platforms shares compute between deadline-critical RAN functions and other workloads. YinYangRAN runs 5G PHY and ML workloads together on GPU accelerators [19], CloudRIC shares pools of different accelerators among distributed units [20], and Concordia reuses idle vRAN CPU capacity while meeting processing deadlines [7]. Other work places AI and RAN workloads through MIG partitions, mixed hardware, and worker location [3, 12, 21, 22], while ARCHES switches PHY models, OCUDU exposes timing classes inside the DU, and SMEC coordinates RAN and edge SLOs [16, 18, 25].

Our work shares the goal of reusing RAN compute for other workloads. However, these systems schedule resources or queued jobs, and their public designs do not represent the recovery that a neural receiver may need. Backstop fills this gap by tracking each pending recovery and by combining one all-fail recovery schedule across cuPHY instances with an AI slot that stays owned until the GPU finishes.

Real-Time GPU Scheduling and Backup Reservation. Earlier work on GPU scheduling makes the latency of concurrent GPU work predictable through microsecond-scale preemption [9], preemptive scheduling across accelerators [23], space and time sharing for real-time DNN inference [4], and hardware partitioning of GPU compute [5]. Fault-tolerant real-time scheduling reserves a backup copy for each primary task, frees the backup when the primary succeeds, and overlaps the backups of primaries that cannot fail together [8]. Imprecise computation splits a task into mandatory and optional parts and runs the optional part only when time remains [13].

Our work builds on these reservation and preemption techniques. However, preemption and partitioning control kernels that are already queued, and overlapping backups assumes that primaries do not fail together. Backstop fills this gap by checking one all-fail schedule for the recovery of every pending receiver, releasing a recovery when its receiver succeeds in time, and enforcing that schedule through GPU launch, buffer ownership, and GPU completion.

Scope of This Comparison. Across the public designs we reviewed through September 2026, we found no system that combines an executable all-fail schedule for the pending recovery of each TB, AI slots held until the GPU finishes, and a prediction of admissible states that uses only validated

bounds. This is a finding about the literature we reviewed, not a claim about unpublished work or work that uses other terms.

6 Limitations

Claim Boundary. The guarantee holds only under the validated bounds and measurement conditions of the evaluated mode. Backstop does not claim that MPS gives hard isolation, that measured maxima are WCETs, or that our test timeline is a production HARQ deadline. Five lifecycle subsets are validated or partly validated and five remain UNVALIDATED; the claim does not require or imply a 10/10 lifecycle matrix.

Test Timeline. P180/D155 is a test timeline: P is the time between TB arrivals, not a 5G slot length, and D155 is not a production HARQ deadline. A builder and checker for production d_MAC traces pass 24 unit tests, but no trace from a target DU/FAPI stack is available. A new timeline requires revalidation on GPUs; we make no production HARQ guarantee.

Production Timing. Aerial testMAC expects an uplink indication with the CRC result by $T_0 + 4.5$ ms. Without AI and with warm workers, three simple paths, which wait for NeuralRx and then recover, decode speculatively on the same GPU, or precompute across GPUs, finish 0/1,000 within 4.5 ms. Moving channel estimation through CRC to GPU1 with raw-IQ P2P reaches 852/1,000, and a shared cuPHY/TensorRT stream reaches 885/1,000. The fixed long-running, stream-ordered fast path reaches 995/1,000 with a 3.813 ms median and a 3.995 ms p99, but five late completions fail the threshold, and the held-out runs stay closed. A 300-request profile traces the remaining tail to remote channel estimation: its GPU time has a p50/p99/max of 0.866/4.873/9.592 ms, and its correlation with the end-to-end pair time is 0.9603. P2P copies average 36.261/17.150 μ s, derate matching exposes a 1,060.105 μ s host call despite a 0.168 ms GPU p99, TensorRT enqueue averages 7.443 μ s, and the TensorRT graph has a 0.886 ms GPU p99. These profiles are mechanism evidence rather than validation or WCET. TestMAC is a controlled developer L2, not a source of field-DU d_{MAC} timing.

Channel Scope. External Aerial TDL-A remains unsupported: across raw and normalized inputs, public reference radio settings, one-antenna channel execution in frequency and time, and an FP32 engine built like the public notebook, conventional decoding succeeds in every ten-TB test and NeuralRx succeeds in none. A fixed development grid selects Sionna CDL-D/E at 100 ns and keeps the failures of CDL-A/B/C, and held-out runs with separate seeds use five Es/No levels and 50 blocks per level and model. Both receivers pass 50/50 at 10 dB. In CDL-D, the conventional receiver decodes 153/250 blocks and NeuralRx 164/250, with 16 blocks decoded only by NeuralRx and five only by the conventional receiver; in CDL-E, the counts are 155/250 and 163/250, with 15 and seven. The 31 neural-only versus 12 conventional-only outcomes give a two-sided exact $p = 0.00540$: optional NeuralRx has conditional radio value in the fixed Sionna CDL-D/E mode. That channel has not passed production timing or end-to-end validation.

Finite Samples. Zero observed violations prove neither a WCET nor a zero failure probability. Bounds belong to one combination of hardware, software, placement, workload, and lifecycle. All main hardware results use A100 GPUs, and the NVLink results do not carry over to other GPU families or to data paths between nodes.

Unresolved Long Tail. In one development attempt, one NeuralRx completion took 350.948 ms while the p99 of the 400-request run was 8.111 ms; the run had 4/400 deadline misses. Targeted campaigns with idle periods, MPS restarts with revalidation, Qwen reloads, and reconnects did not reproduce it. Its cause is unknown, and GPU and driver hangs remain outside the fault model.

Workload Mapping. BurstGPT supplies real arrival times and prompt sizes, and Qwen2.5-1.5B prefill stands in for external AI work. The prompt-size classes and AI deadlines are our experimental choices; other workloads need new bounds and a new validation.

7 Conclusion

In this paper, we present Backstop, a recovery-safe GPU scheduler that shares AI-RAN GPUs between neural receivers and AI inference. By tracking each pending recovery and admitting AI only when an all-fail schedule still fits, Backstop keeps radio recovery within its deadline while it reuses the GPU time that neural-receiver successes free. Our evaluation shows that Backstop reaches the best possible AI capacity at 13,716 feasible model settings and delivers a median of 1.44× the timely AI value of recovery-first at the trace settings where order matters. On two four-GPU nodes, Backstop has 0/120 safety violations in boundary tests where idle-time admission fails 40/40, serves 6.2-18.5% more on-time AI tokens than recovery-first under bursty AI arrivals, and commits every TB exactly once under seven kinds of injected faults. On one GPU, early AI slots add 2.6-70.3% more on-time AI tokens without losing a timely NeuralRx result. These results validate a test timeline, not production HARQ timing.

References

- [1] 3GPP. 2020. *Study on Channel Model for Frequencies from 0.5 to 100 GHz*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3173> Technical Report 38.901.
- [2] 3GPP. 2024. *NR; Physical Layer Procedures for Data*. 3rd Generation Partnership Project. <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3216> Technical Specification 38.214.
- [3] Ganesh Ananthanarayanan, Matthew Balkwill, Xenofon Foukas, Zhihua Lai, Bozidar Radunovic, Connor Settle, and Yongguang Zhang. 2025. Distributed AI Platform for the 6G RAN. In *ACM Workshop on Open and AI RAN*. ACM, New York, NY, USA, 15–21. <https://doi.org/10.1145/3737900.3770167>
- [4] Amir Fakhim Babaei and Thidapat Chantem. 2025. DARIS: An Oversubscribed Spatio-Temporal Scheduler for Real-Time DNN Inference on GPUs. In *ACM/IEEE Design Automation Conference*. IEEE, Piscataway, NJ, USA, 1–7. <https://doi.org/10.1109/DAC63849.2025.11132423>
- [5] Joshua Bakita and James H. Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *ECRTS*, Vol. 335. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Wadern, Germany, 21:1–21:25. <https://doi.org/10.4230/LIPIcs.ECRTS.2025.21>
- [6] Sebastian Cammerer, Fayçal Ait Aoudia, Jakob Hoydis, Andreas Oeldemann, Andreas Roessler, Timo Mayer, and Alexander Keller. 2023. A Neural Receiver for 5G NR Multi-User MIMO. In *IEEE Globecom Workshops*. IEEE, Piscataway, NJ, USA, 329–334.
- [7] Xenofon Foukas and Bozidar Radunovic. 2021. Concordia: Teaching the 5G vRAN to Share Compute. In *ACM SIGCOMM*. ACM, New York, NY, USA, 580–596. <https://doi.org/10.1145/3452296.3472894>
- [8] S. Ghosh, R. Melhem, and D. Mossé. 1997. Fault-Tolerance Through Scheduling of Aperiodic Tasks in Hard Real-Time Multiprocessor Systems. *IEEE Transactions on Parallel and Distributed Systems* 8, 3 (1997), 272–283. <https://doi.org/10.1109/71.584093>
- [9] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-Scale Preemption for Concurrent GPU-Accelerated DNN Inferences. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>
- [10] Mikko Honkala, Dani Korpi, and Janne M. J. Huttunen. 2021. DeepRx: Fully Convolutional Deep Learning Receiver. *IEEE Transactions on Wireless Communications* 20, 6 (2021), 3925–3940. <https://doi.org/10.1109/TWC.2021.3054520>
- [11] Jakob Hoydis, Sebastian Cammerer, Fayçal Ait Aoudia, Avinash Vem, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. 2022. Sionna: An Open-Source Library for Next-Generation Physical Layer Research. arXiv:2203.11854 <https://arxiv.org/abs/2203.11854>
- [12] Haiyuan Li, Yulei Wu, and Dimitra Simeonidou. 2026. Deadline-Driven Hierarchical Agentic Resource Sharing for AI Services and RAN Functions in AI-RAN. arXiv:2605.07547 [cs.NI] <https://arxiv.org/abs/2605.07547>
- [13] Jane W.-S. Liu, Kwei-Jay Lin, W. K. Shih, Albert Chuang shi Yu, Jen-Yao Chung, and Wei Zhao. 1991. Algorithms for Scheduling Imprecise Computations. *Computer* 24, 5 (1991), 58–68. <https://doi.org/10.1109/2.76287>
- [14] NVIDIA Corporation. 2026. *Aerial CUDA-Accelerated RAN Documentation*. NVIDIA. <https://docs.nvidia.com/aerial/cuda-accelerated-ran/>
- [15] NVIDIA Corporation. 2026. *Multi-Process Service Documentation*. NVIDIA. <https://docs.nvidia.com/deploy/mps/>
- [16] Timothy O'Shea, Matthew Pennybacker, and Andriy Kharchenko. 2026. The OCUDU dApp Platform: An Open Runtime and E3 Interface for Real-Time AI-RAN. arXiv:2609.07843 [cs.NI] <https://arxiv.org/abs/2609.07843>
- [17] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 <https://arxiv.org/abs/2412.15115>

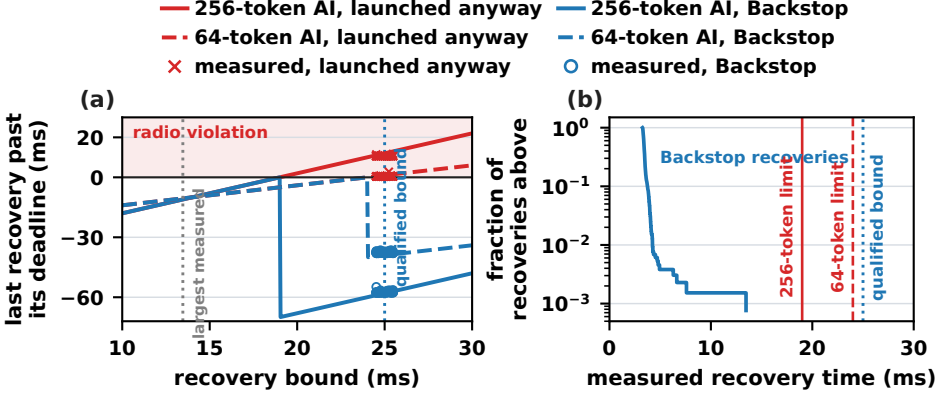


Fig. 20. How the recovery bound changes the violation. (a) How far the last recovery ends past the recovery deadline when the request that Backstop rejects runs anyway, against the recovery bound B_{conv} : lines follow the schedules of the 256-token case (E4) and the 64-token case (E6b) at their full bounds, and markers show the physical attempts measured at the validated 25 ms bound. (b) Share of the 1,312 measured recovery times of the two-node campaign that are longer than a given length, with the bound at which each case disappears (limits) and the validated bound.

- [18] Neagin Neasamoni Santhi, Davide Villa, Michele Polese, Salvatore D'Oro, Yunseong Lee, Koichiro Furueda, and Tommaso Melodia. 2026. ARCHES: Adaptive Real-Time Switching of AI Models for the RAN. arXiv:2604.23397 [cs.NI] <https://arxiv.org/abs/2604.23397>
- [19] Leonardo Lo Schiavo, Jose A. Ayala-Romero, Andres Garcia-Saavedra, Marco Fiore, and Xavier Costa-Perez. 2024. YinYangRAN: Resource Multiplexing in GPU-Accelerated Virtualized RANs. In *IEEE INFOCOM*. IEEE, Piscataway, NJ, USA, 721–730. <https://doi.org/10.1109/INFOCOM52122.2024.10621380>
- [20] Leonardo Lo Schiavo, Gines Garcia-Aviles, Andres Garcia-Saavedra, Marco Gramaglia, Marco Fiore, Albert Banchs, and Xavier Costa-Perez. 2024. CloudRIC: Open Radio Access Network (O-RAN) Virtualization with Shared Heterogeneous Computing. In *ACM MobiCom*. ACM, New York, NY, USA, 558–572. <https://doi.org/10.1145/3636534.3649381>
- [21] Syed Danial Ali Shah, Maryam Hafeez, Abdelaziz Salama, and Syed Ali Raza Zaidi. 2025. Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks. arXiv:2507.09124 [cs.NI] <https://arxiv.org/abs/2507.09124>
- [22] Syed Danial Ali Shah, Zeinab Nezami, Maryam Hafeez, and Syed Ali Raza Zaidi. 2025. The Interplay of AI-and-RAN: Dynamic Resource Allocation for Converged 6G Platform. In *IEEE INFOCOM Workshops*. IEEE, Piscataway, NJ, USA, 1–6. <https://doi.org/10.1109/INFOCOMWKSHPs65812.2025.11152939>
- [23] Weihang Shen, Mingcong Han, Jialong Liu, Rong Chen, and Haibo Chen. 2025. XSched: Preemptive Scheduling for Diverse XPU. In *USENIX OSDI*. USENIX Association, Boston, MA, USA, 671–692. <https://www.usenix.org/conference/osdi25/presentation/shen-weihang>
- [24] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yehu Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *ACM SIGKDD*. ACM, New York, NY, USA, 5831–5841. <https://doi.org/10.1145/3711896.3737413>
- [25] Xiao Zhang and Daehyeok Kim. 2026. Enabling SLO-Aware 5G Multi-Access Edge Computing with SMEC. In *USENIX NSDI*. USENIX Association, Boston, MA, USA, 1759–1776. <https://www.usenix.org/conference/nsdi26/presentation/zhang-xiao>

A Sensitivity to the recovery bound

Recovery Bound. The boundary cases of Section 4.7 use the validated bound $B_{\text{conv}} = 25$ ms. Figure 20a shows how the overrun of a rejected request depends on this bound. The overrun of E4 grows by 2 ms and that of E6b by 1 ms per millisecond of bound because each needed recovery adds its bound to the end of the last recovery, and the physical attempts measured at the validated

bound lie within 1.8 ms of these lines. E6b disappears at or below 24 ms, while E4 remains until 19 ms. Figure 20b shows that the 1,312 measured recovery times have a median of 3.404 ms and a maximum of 13.465 ms, below both limits. A measured maximum cannot replace a validated bound; lowering the bound defines another mode that needs revalidation, and Proposition 2 still gives a later decision time at which ignoring pending recovery is unsafe, for every positive bound. Therefore, the padded campaign shows that the admission check is necessary under the declared bounds; it is not a claim about production rates or WCET.